ListMerger: A JavaScript Component for Manual List Merging

Martin Rabensteiner

706.414 Seminar/Project Interactive and Visual Information Systems 4SP WS 2025/2026
Graz University of Technology

17 Apr 2026

Abstract

The ListMerger is a TypeScript-written JavaScript library to facilitate the merging of multiple lists. An application example is the merging of findings from multiple evaluators in a heuristic evaluation. The library bases primarily on modern usage of HTML, CSS, and JavaScript, and largely avoids third-party plugins. This paper contains a description of the implementation, a brief introduction into the techniques and use cases, and user guides for developers, implementors, and end-users.

© Copyright 2026 by the author, except as otherwise noted.

This work is placed under a Creative Commons Attribution 4.0 International (CC BY 4.0) licence.

Contents

Contents ii

List of Figures iii

List of Tables v

List of Listings vii

1 Introduction 1

2 List Merging 3
- 2.1 Acquisition Lists . . . 3
- 2.2 Recipes . . . 3
- 2.3 Heuristic Evaluations . . . 3
- 2.4 Related Work . . . 4

3 Modern Web Technologies 7
- 3.1 Living HTML . . . 7
- 3.2 Modern CSS . . . 7
- 3.3 Modern JavaScript . . . 7
- 3.4 Node.js . . . 8
- 3.5 jQuery . . . 8
- 3.6 JavaScript Package Managers . . . 8
- 3.7 GitHub Pages . . . 8
- 3.8 Mustache Templates . . . 8
- 3.9 Accessibility . . . 9

4 ListMerger 11
- 4.1 Files . . . 11
- 4.2 Data Structure . . . 11
- 4.3 History (Undo/Redo) . . . 11
- 4.4 Initialisation . . . 12
- 4.5 Moving Items . . . 12
- 4.6 Styles . . . 12
- 4.7 Accessibility Mode . . . 12
- 4.8 Cursor Behaviour when Dragging . . . 12
- 4.9 Templates . . . 12
- 4.10 Tab Bar . . . 13
- 4.11 Icons . . . 14

A End User Guide **15**

B Library User Guide **17**

Bibliography **19**

List of Figures

2.1 Sketch of a Recipe Overview 4
2.2 HEM Findings Merger 5
2.3 Heurio: Kanban board 5

4.1 Tabs 14

List of Tables

4.1 Icons 14

List of Listings

4.1 Tab Bar CSS 13

B.1 JSON Item Lists 18

Chapter 1

Introduction

This report describes the purpose, development, and usage of a tool for list merging. The use cases are further described in Chapter 2 and the development in Chapter 4. Finally, [TODO:] .

Chapter 2

List Merging

List merging as a term describes the process to merge the items of several lists into one aggregated list. This chapter covers use cases and the affordance of list merging. Further, implementation examples and related work is described.

2.1 Acquisition Lists

Different departments or employees can enter their needs and wishes for supplies. An acquisition manager has then an overview of all wishes. Identical or similar items can be combined and the quantity of the concerning item increased.

2.2 Recipes

Recipes would be use case where similar items in different lists would be beneficial and desirable. The origin lists are for recipes selected by the prospective cook. In the merged list, the needed or the already available ingredients can be gathered. This highlights overlapping ingredients in recipes and helps to depict which recipes can be cooked with the available ones or which ones need to be bought. A sketch of this example can be seen in Figure 2.1.

2.3 Heuristic Evaluations

The heuristic evaluation is a form of usability inspection, first described by Nielsen and Molich [1990]. A small number of usability experts, usually 3 to 5, review a user interface and use their judgement and experience to find potential problems. The evaluation is guided by a small set of broad principles (the heuristics) for good user interface design. Outside academic circles, the technique is often known as an *expert review*. Andrews [2025] describes the ideal process of a heuristic evaluation with five phases: Preparation, Individual Evaluations, Aggregation, Severity Ratings, and Debriefing and Report. The essential steps of list merging are the second and third. In the Individual Evaluations phase, the evaluators observe the interface completely independent and described the issues found by them. In the Aggregation phase afterwards, an evaluation manager needs to aggregate these findings and combine similar ones. Therefore, the evaluation manager must be able to simple combine the issues with all their metadata (heuristic, description, attached screenshots, and so on) Those are then further rated according to their severity and described in a report.

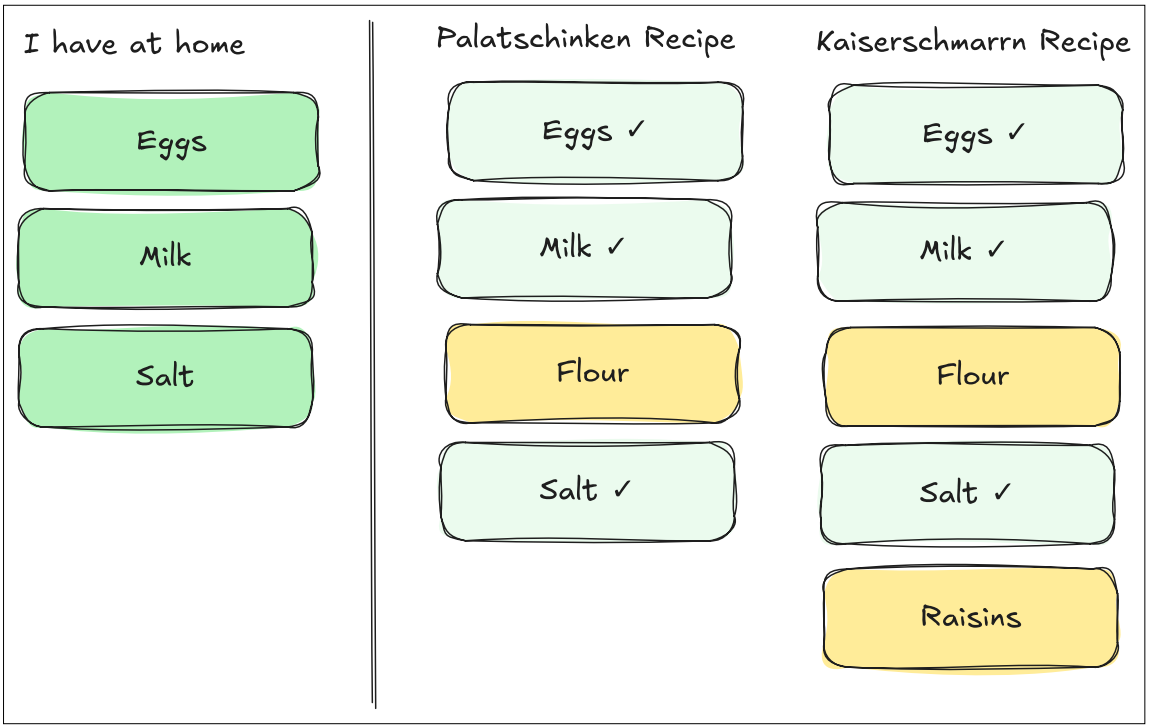

Figure 2.1: In this example, the ingredients of recipes that are already to hand can be put on the merge list, to emphasise ingredients that need to be bought first. [Sketch by Martin Rabensteiner.]

2.4 Related Work

For the last mentioned application case, Loitzl [2006] introduces within the Heuristic Evaluation Manager (HEM) a merging interface to merge usability issues, as seen in Figure 2.2. Findings can be selected for merging with the + button. Red indicates currently selected findings. Darkgrey findings are already added to the merge list. This is done with inline JavaScript calls by pressing a button, and without drag-and-drop functions.

The functionality of Kanban boards, like the issue tracking app Heurio in Figure **??** seems similar. But the columns represent different stages or filters, and the possibility to merge is not designated.

By researching, no equivalent libray, nor applications with dynamic, visual merging could be found. Therefore it was decided to create ListMerger with its implementation and functionality further described in the following chapters.

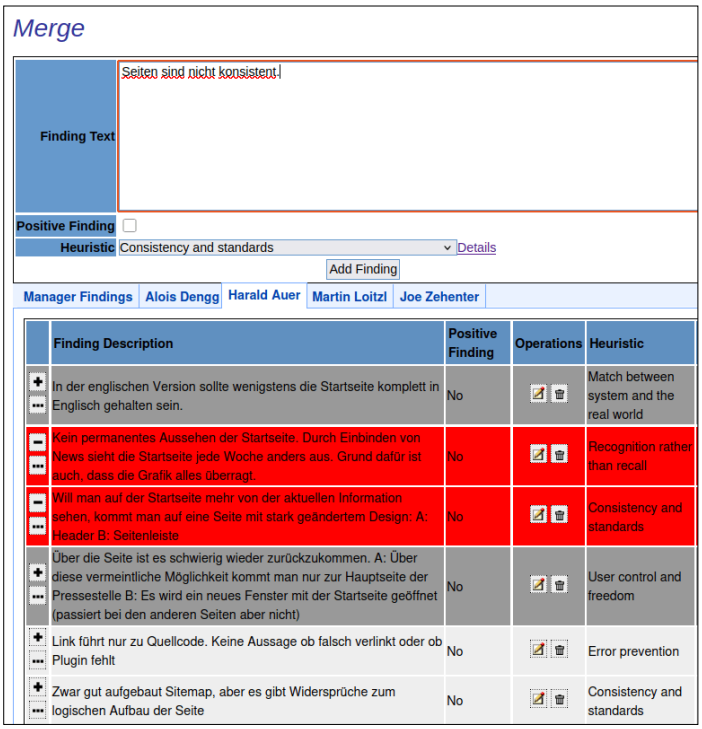

Figure 2.2: The findings merger in HEM: Findings can be selected for merging with the + button. Red indicates currently selected findings. Darkgrey findings are already added to the merge list. [Screenshot by Martin Rabensteiner.]

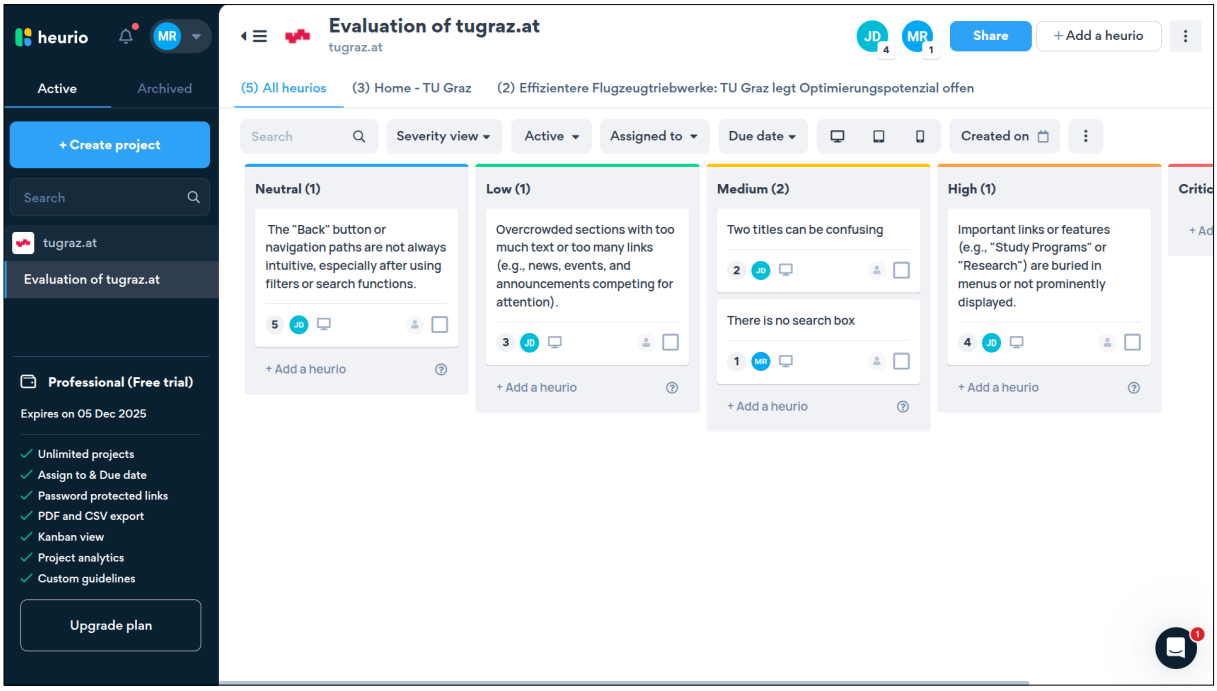

Figure 2.3: Usability findings in Heurio are by default organised as a Kanban board and sorted by severity. [Screenshot by Martin Rabensteiner.]

Chapter 3

Modern Web Technologies

[TODO:]

The main browser vendors united already in 2004 as Web Hypertext Application Technology Working Group (WHATWG), but the definition of the HTML standard was in charge of the World Wide Web Consortium (W3C) for some decades. In 2018, the browser vendors became mutinous and only one year later, both signed an agreement where W3C is giving the vendors more say [Cimpanu 2019]. This led to more standardization among different browsers, and a faster drive of innovation, like the improvements mentioned as follows.

3.1 Living HTML

The Hyper Text Markup Language (HTML) standard was first developed and published in the early 1990's by Tim Berners-Lee at CERN and later handed over to the new founded World Wide Web Consortium (W3C) [WHATWG 2026]. The last major version (HTML4) was published in 1998, and after some intermediate versions, the work on HTML5 started by W3C in 2004. HTML5 was never published in a finished version. Since 2019, W3C and WHATWG are maintaining the standard together as a living standard, and publish updates continuously in the publishing browers.

3.2 Modern CSS

Along with the HTML modernisation, the Cascading Style Sheets (CSS) became a living standard as well [W3C 2026]. CSS is used to define to optical style of a website. Modern solutions include for example containers queries [MDN 2026b] to differentiate between different screen sizes or use styles as selectors, or colour mixing functions [MDN 2026a].

3.3 Modern JavaScript

[Shukla 2023]

JavaScript is essential in modern web development to make client-site computations and modify the Document Object Model (DOM), which describes the structure and content of a web page. It was introduced in 1995, soon available in all usual browsers, and is and still today an important component of them. As dynamic web pages become more and more popular, it was even more necessary to manipulate the content directly on the client.

3.4 Node.js

With Node.js, it became popular to use JavaScript also as a server-side programming language.

3.5 jQuery

jQuery is a widley-used frontend framework and reached its peak of usage on around 78 % of all websites tracked in 2022 and is steadly decreasing since then, with circa 70 % in March 2026 [W3Techs 2026]. By time, the outstanding functions of jQuery got more and more redundant. Several use cases and functionalities were integrated directly in HTML, CSS, or JavaScript. For example, the accordion can now be replaced with the `<details>` tag (available since 2020) [MDN 2025a]. JavaScript introduced selector functions like `querySelector()`, `querySelectorAll()` and some other for HTML element in 2015 [MDN 2025c]. This made the classic jQuery DOM selection with the `\$()` function obsolete.

3.6 JavaScript Package Managers

The node package manager (NPM) was introduced in 2009, to make the handling with node modules in the development process and their publishment among developers more convenient. It is an open source project and now in the hands of GitHub [NPM 2026].

PNPM (performant NPM) introduces a more performant approach for package managing than NPM PNPM 2026. Instead of downloading and storing dependencies for every project on its own, they are managed globally on the system, and symbolic links are created from the project to the main storage. That saves time and storage space. Further, PNPM is capable of managing different versions of Node. For other package managers, a separate Node Version Manager is needed. Disadvantages arise when multiple disks are used on a system, as the symlinks used by PNPM are not capable of that.

To measure the popularity of package managers, Maj [2024] discusses different ways to count the usage. Different ways of downloads have the problem of different sources, and packager managers with more frequent updates will have higher download counts. The observation of the most popular GitHub repositories is considered a meaningful measurement. According to the last iteration Maj 2026 and compared with the last three years, NPM is steadily used in about two third of projects. The usage of Yarn classic has decreased from 29 % to 18 % over the same time frame. PNPM has a stagnant share of about 6 %.

PNPM is used in this project to combine its advantages in speed and storage performance with

3.7 GitHub Pages

[TODO:]

3.8 Mustache Templates

Mustache is a logic-less template format [Mustache 2024]. Characteristic are the double `\{\{\}\}` brackets, which are then replaced. For example, `\{\{variable\}\}` is replaced with the value of `variable`. For-each loops and simple if conditions can be mocked with Mustache, even though there is no logic in the template. Mustache libraries are available for around 50 programming languages, including JavaScript, Python, PHP, or C++. It is popular to structure small or large parts of content with its template style.

3.9 Accessibility

Chapter 4

ListMerger

[TODO:]

This section describes the implementation of ListMerger. The code itself is written in pure TypeScript. Almost all third-party librariers are only used in the build and development process. Only the Mustache package is used to render templates.

Recurring terms in the development are move and merge. Moving describes the process of a transferring an item from A merged item always consists of two predecessors. They are either taken from the mergelist (merged or moved), or directly taken from the evaluators list. The origin is always tracked, independent whether The differentiation between moved and origin is done, to have a clean and reconstructive merging history. When saving, the order of the merged elements is taken from the list, while the data about merged is stored in an array.

4.1 Files

The TypeScript code is modularised into several files, each serving a different purpose. The files are:

- `events.ts`: Initialises all event listeners for buttons and drag events, and contains their functions.
- `globalvars.ts`: Contains some shared variables used across several files.
- `history.ts`: Tracks all actions and recovers them when the Undo or Redo button is pressed.
- `index.ts`: Serves only as the interface between the module and the outer implementation.
- `init.ts`: Contains the only function callable from the outside.
- `transfer.ts`: Handles the transfers between the source lists and the merged list.
- `uihelper.ts`: Contains several helper functions to deal with the DOM.

4.2 Data Structure

When initialising, a JavaScript object has to be handed over to the library in order to build the application. More details on this initialisation can be found in Appendix B.

4.3 History (Undo/Redo)

An important feature and affordance of this component is the history tracking. With the Undo and Redo buttons, the user can jump back and forward with actions.

The history is based on a history and a future array. When clicking undo/redo, a history item is moved from history to future, or vice versa. A history item consists of a keyword for the action, and three parameters. The parameters are strings, and can be IDs, or positions, depending on the context of the merge keyword.

4.4 Initialisation

A detailed guide to the initialisation of a ListMerger instance can be found in Appendix B. [TODO:]

4.5 Moving Items

[TODO:]

4.6 Styles

The ListMerger itself does not define any styles. These are entirely up to the individual implementation. Only the base-structure of the application is to a certain extent given by the library itself. The demo files can be used as a staring point or completely

4.7 Accessibility Mode

WCAG 2.2 Success Criterion 2.5.7 requires alternative methods

To activate this accessibility mode, the `toggleAccessibilityMode()` function needs to be called. This can be done when initialising the library with a `toggleAccessibilityMode(true)` call, to always initalise it in the beginning. When toggling is afforded, the function without a parameter can be bound to a button.

The accessibility does not affect sizes, colors, or other visual properties of the page, since the are entirely depending on the implementation on the page and the custom CSS.

4.8 Cursor Behaviour when Dragging

The `event.datatransfer.dropEffect` property in JavaScript defines the visual behaviour of the mouse cursor during a drag event [MDN 2025b]. While the `cursor` CSS property offers a variety of 36 mouse cursors, in interaction with the operating system [MDN 2026c], this property is not taken into account as soon as a drag event is started. The `dropEffect` has offers a selection of only four elements, all equivalent to options for the CSS property.

4.9 Templates

To make ListMerger as flexible as possible, some element specifications are placed in the hands of templates. For this, the Mustache template engine is used (see Section **??**). The templates used are for three parts. Those are:

- *Item*: The appearance of a single item in the origin list, or the merged one.
- *Dialog*: When set, a dialog opens when clicking on an item.
- *Merge*: The middle column when open in the merge screen. If the `name` attribute of an Input or TextArea field is ident to a value of an item, it is buttons are shown to add the value

```
.container {
  display: flex;
  flex-wrap: wrap;
}

details {
  display: contents;
}

details[open]::details-content {
  display: contents;
}

summary {
  order: 0;
}

section {
  order: 1;
  width: 100%;
}
```

Listing 4.1: The minimal CSS styling to use the `<details>` element and its children as a tab bar.

4.10 Tab Bar

For a long time, it was necessary to use JavaScript event listeners for tab bars in the web. By adding click event listeners to the title, the appearance of the content below in that way can be controlled. The `<details>` HTML element and some CSS adjustments make it possible to have tab bars without JavaScript.

The `<details>` element serves as a container for one item. On `<summary>` element is needed as a child, which is represents the title. If no such title element is present, the total is Details. Per default, a small triangle on the left indicates the open/closed status of the content. This status is toggled by clicking on the `<summary>` element. The content does not necessarly have to be an element and can also be plain text. To have a clear structure, it is still advisable to have an element to give the content a better structure, for example with a `<section>` element. With the `open` attribute, the fold out of `<details>` can be controlled. The `name` attribute can be used to only open one `<details>` box when having several instances.

The original purpose of this element is to build Accordions. To use it for tab bars, some CSS properties need to be applied. These properties include:

- *`<container>` as flexbox*: This ensure to have the elements next to each in horizontal alignment.
- *`<section>` with full-width (100%)*: Below the flex container, the summaries should have only as much width as they need, but the section below should span across the whole width to break in a new line.
- *Display details as content*: The section would be shown right next to the summary, but this display option shifts it to the end.
- *Display open details pseudo element as content*: This pseudo element
- *Order of summary and section*: Setting the order for all summary items to 0 and all section items to

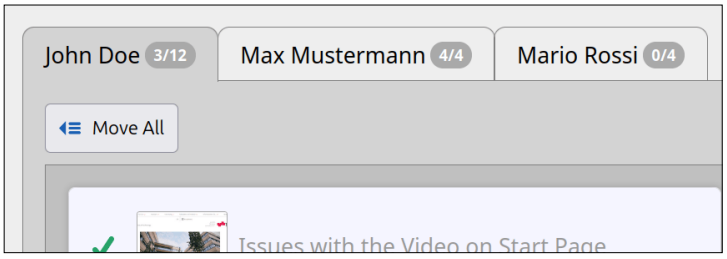

Figure 4.1: The tabs in ListMerger show the title of each list and an indicator with the number of added/merged and total items in the list. [Screenshot by Martin Rabensteiner.]

Icon	Filename	Description
	`save.svg`	Save the current state.

Table 4.1: Overview of icons made for and used within the library.

1, clusters the summary items all at the top and shows the active section element below.

The minimal CSS to reach this appearance in shown in Listing 4.1. Additionaly to that, the tabs are also styling with some basic CSS, including, colors, borders, border-roundings, margins, paddings, and the cursor, as seen in Figure 4.1.

However, to maintain responsiveness the solution in ListMerger still uses JavaScript to transform the bar of tabs in a selection box on smaller screens or with longer titles. A `<select>` box holds the titles as values and the order as values. The value of the box needs to be tracked and updated as well when the open tab changes. It also needs to update when elements are called with a hash in the URL.

A `ResizeObserver` instance helps to track the size or position of an element, changed for example through scrolling or resizing the window. In this case it is used to observe the position of the `<summary>` elements, representing the titles of the tabs. If the titles are too long for the viewport, the titles would break and be represented in more than one line. This is not a suitable solution for tabs and they should be controlled by a select box, as described before. Within the observer, the top offset of the first and last object are compared, and if they are not the same, the responsive solution should be applied. A CSS class (`.compact` in the example) is added to the tab list. The titles can not be simply hidden, as this would give the observer an undetermined width and lever its functionality. Therefore, the titles are made transparent, paddings removed, and the height set to `1px`. This makes it practically invisible, but leaves the observer functionality intact.

4.11 Icons

Especially the merging icon made it necessary to work out custom icons for this project. The icons were initally drafted in Inkscape and the resulting SVG files manually revised and improved. All icons base on a 50 x 50 viewbox. Strokes have usually a width of 12 and a round linejoin. All icons are listed in Table 4.1. The iconset does not have to be used, but can also be replaced or hidden completely.

Appendix A

End User Guide

End users can switch between separated list via tabs, or on more narrow screens with a select box. Items can be dragged and dropped from one list on the right to the mergelist on the left. An item is inserted at the location where it is dropped. The Move All button can be used to move all existing items of a list to the merged list. The appearance of items in the list, the info dialog, and the merge dialog is dependent on the individual implementation.

By dragging one item over another in the mergelist, those items can be merged. With the drag handle on the right, they can be rearranged. Undo and Redo let the user jump through the history. The behaviour of the Save button is given by the individual implementation.

Next to the tab titles are small numbers that indicate the current state of the list. For origin lists, they count how much of items have been added or merged into the mergelist out of the total number per list. For the mergelist, the indicator shows the total number of items in the list, not necessarly how much have been added.

Appendix B

Library User Guide

This section addresses to those that want to include the ListMerger library in a larger software project. To initialise a page with lists, the `init.init()` function needs to be called. The parameters contain an overall HTML id, the ids for the Undo and Redo buttons, a function that can be called when the Save button is clicked, an items object, and several strings for HTML templates. A full example of a configuration can be seen in the `example/main.js` file.

Items have to be handed over in an object that needs to contain at least a `merged` key and an `originlists` key. The first can be an empty array, or contain items. The second one needs to be filled with the lists as objects with some metadata (`id`, `name`, etc.) and an `items` key, which also contains an array of items, as seen in Listing B.1.

An item needs to have at least an `id`, this id is also used as id in the HTML DOM. The other keys are optional and need to be included depending on whether they are needed in the user-defined templates. Arrays (like images) are always merged together.

Templates define how the items appear in the lists (`item_template`), their info modals (`dialog_template`), and the form to merge two items (`merge_template`). In the merge form, the `name` attribute needs to be ident with the keys used in an item. A callback function can be handed over to process the data further, for example to store it in a database. This function is then called with an array of moved, and merged items. In the example, the data just is written to the JavaScript console.

```
{
  "merged": [],
  "originlists": [
    {
      "id": "eval1",
      "name": "List 1",
      "items": [
        {
          "id": "li1it1",
          "title": "Item 1",
          "author": "Martin Rabensteiner",
          "category": "Category 1",
          "images": [
            "images/screenshot1.png",
            "images/screenshot2.png"
          ],
          "description": "This is the first item in the list."
        }
      ]
    }
  ]
}
```

Listing B.1: Example of an initialising object for items. The items in `merged` are rendered to the mergelist, `originlists` are constructed as lists with the base items.

Bibliography

Andrews, Keith [2025]. *Human Computer Interaction: Lecture Notes*. 27 May 2025. http://courses.isds.tugraz.at/hci/hci.pdf (cited on page 3).

Cimpanu, Catalin [2019]. *Browser Vendors win War with W3C over HTML and DOM Standards*. 28 May 2019. https://zdnet.com/article/browser-vendors-win-war-with-w3c-over-html-and-dom-standards/ (cited on page 7).

Loitzl, Martin [2006]. *The Heuristic Evaluation Manager (HEM): An Online Collaborative Environment for Heuristic Evaluation*. Master's Thesis. Institute for Information Systems and Computer Media (IICM): Graz University of Technology, Feb 2006. 228 pages. https://ftp.isds.tugraz.at/pub/theses/mloitzl.pdf (cited on page 4).

Maj, Wojciech [2024]. *Package Manager Wars. The Real Picture*. 26 Jan 2024. https://dev.to/wojtekmaj/package-manager-wars-the-real-picture-e9p (cited on page 8).

Maj, Wojciech [2026]. *Repository: Package Manager Stats*. 13 Mar 2026. https://github.com/wojtekmaj/package-manager-stats (cited on page 8).

MDN [2025a]. *<details>: The Details Disclosure Element*. 13 Oct 2025. https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/details (cited on page 8).

MDN [2025b]. *DataTransfer: dropEffect property*. 02 Oct 2025. https://developer.mozilla.org/en-US/docs/Web/API/DataTransfer/dropEffect (cited on page 12).

MDN [2025c]. *Document: querySelector() Method*. 07 Nov 2025. https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelector (cited on page 8).

MDN [2026a]. *CSS: Color()*. 07 Nov 2026. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/color (cited on page 7).

MDN [2026b]. *CSS: Container Queries*. 24 Mar 2026. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_queries (cited on page 7).

MDN [2026c]. *CSS: Cursor*. 07 Jan 2026. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/cursor (cited on page 12).

Mustache [2024]. *Mustache: Logic-Less Templates*. 09 Aug 2024. https://mustache.github.io/ (cited on page 8).

Nielsen, Jakob and Rolf Molich [1990]. *Heuristic Evaluation of User Interfaces*. Proc. SIGCHI Conference on Human Factors in Computing Systems (CHI 1990) (Seattle, Washington, USA). ACM. 01 Apr 1990, pages 249–256. doi:10.1145/97243.97281. https://dl.acm.org/doi/pdf/10.1145/97243.97281 (cited on page 3).

NPM [2026]. *NPM Documentation*. 19 Mar 2026. https://docs.npmjs.com/ (cited on page 8).

PNPM [2026]. *PNPM: Fast, Disk Space Efficient Package Manager*. 19 Mar 2026. https://pnpm.io/ (cited on page 8).

Shukla, Abhishek [2023]. *Modern JavaScript Frameworks and JavaScript's Future as a Full-Stack Programming Language*. Journal of Artificial Intelligence & Cloud Computing 144.2 (Oct 2023), pages 2–5. doi:10.47363/JAICC/2023(2)144. https://onlinescientificsresearch.com/articles/modern-javascript-frameworks-and-javascripts-future-as-a-full-stack-programming-language.pdf (cited on page 7).

W3C [2026]. *Cascading Style Sheets*. 14 Apr 2026. https://www.w3.org/Style/CSS/ (cited on page 7).

W3Techs [2026]. *Historical Yearly Trends in the Usage Statistics of JavaScript Libraries for Websites*. 20 Mar 2026. https://w3techs.com/technologies/history_overview/javascript_library/all/y (cited on page 8).

WHATWG [2026]. *HTML - Living Standard*. 14 Apr 2026. https://html.spec.whatwg.org/ (cited on page 7).